

De código legado a una especificación verificada

Scan, Understand, Verify, Deepen, Report y Plan: un paquete de entrega listo para modernizar

El problema: sistemas que nadie entiende ya





10 a 25 años de antigüedad

ASP.NET WebForms, ASP clásico, VB6, Java EE, PHP 5, Delphi, hojas Excel con macros.



Sin documentación fiable

Falta o está desfasada, y quienes lo escribieron ya no están.



Reglas de negocio escondidas

En manejadores de página, procedimientos almacenados, banderas de configuración y cadenas SQL.



Semanas de trabajo manual

Depende de una persona y nadie puede comprobar de dónde sale cada afirmación.

Usar un asistente de IA sin más no basta

Riesgo	Qué ocurre	Respuesta de Rosetta
Salida no verificable	Una regla aparece en el resumen y no existe en el código	Cada afirmación cita líneas y el verifier la comprueba
Cobertura incompleta	Lee lo que cabe en su contexto y omite el resto en silencio	Mapa del código del Scan y cobertura por área
Coste sin control	Sesiones largas con APIs de pago y sin tope	Estimación, reserva y topes
Dependencia y privacidad	Un solo proveedor; el repositorio entero, con secretos, sale a la nube	Proveedor intercambiable; secretos enmascarados
No accionable	Un texto largo que no dice qué hacer ni en qué orden	El Plan entrega decisiones, hoja de ruta y tareas

Qué es Rosetta

- Una aplicación web con cuentas y control de coste.
- Analiza código de cualquier stack; un pack de lenguaje enriquece el mapa.
- Escribe una especificación en la que cada hallazgo cita archivo y líneas.
- Entrega un paquete de traspaso independiente de la herramienta.
- No reemplaza ningún sistema ni reconstruye el legado.



El pipeline, de punta a punta

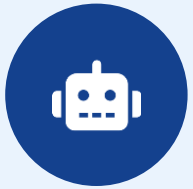
1



Scan

Mapa del código sin IA: archivos, lenguajes, puntos de entrada y áreas. Coste cero.

2



Understand

El reader lee cada área y escribe tarjetas con citas.

3



Verify

Cada cita se comprueba contra sus líneas y el verifier la juzga.

4



Deepen

Vuelve sobre lo que no quedó Supported: relee y juzga de nuevo.

5



Report

Hallazgos, veredictos, preguntas abiertas y coste en un informe.

6



Plan

El planner escribe arquitectura, decisiones, hoja de ruta y tareas.

Las seis etapas están construidas en la release 1 y funcionan en el entorno alojado. Deepen es opcional y se confirma con su estimación.

Agentes en vivo y proveedores intercambiables

Roles de los agentes

reader	lee un área y escribe las tarjetas con sus citas
verifier	juzga cada afirmación contra las líneas citadas
summariser	une tarjetas repetidas y escribe la vista general
planner	escribe el paquete del Plan

La página de la ejecución muestra cada agente, sus tarjetas y el coste por rol mientras trabaja, y la reproduce después.

Un único puerto LlmProvider

- Ollama en local, coste cero
- OpenAI en la nube
- Anthropic en la nube
- Cada rol elige su proveedor y modelo; cada llamada pasa por el guard de egreso y el de presupuesto

Evidencia y verifier: nada se afirma sin citar

Cada afirmación es una tarjeta con al menos una cita ruta:líneaInicio-líneaFin. Nada se borra: lo Rejected se conserva y se muestra con su motivo.



Coste bajo control antes, durante y después

Antes

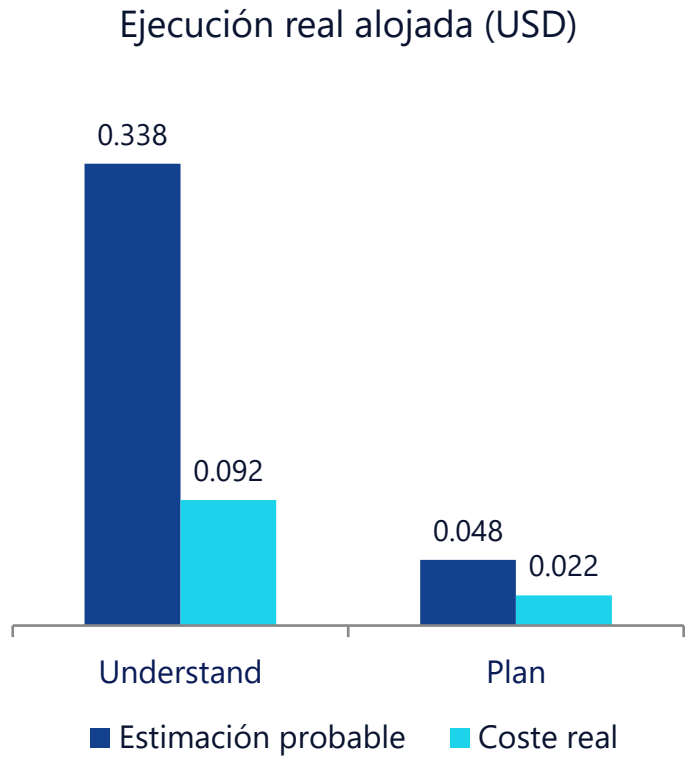
Estimación por rol según la tabla de precios: baja, probable y alta. Se pide confirmación por encima del umbral.

Durante

Cada llamada se reserva por su peor caso; si podría pasar el tope, no se hace. Tokens y coste en vivo.

Siempre

Topes por ejecución, por usuario y día, y por servidor y mes. Los modelos locales cuestan cero y se registran igual.



Seguridad: el código analizado no es de fiar

Solo lectura

Las herramientas de los agentes no escriben; el repositorio analizado nunca se modifica.

Lista de denegados

Se respetan .rosettaignore y los archivos sensibles conocidos.

Secretos enmascarados

Claves, contraseñas y cadenas de conexión se ocultan antes de salir.

Falla cerrado

Si el enmascarado falla, no se envía nada y se registra un evento.

Registro de egreso

Cada extracto enviado a un proveedor queda registrado.

Cuentas y auditoría

Cuentas creadas por el administrador, claves cifradas, accesos auditados.

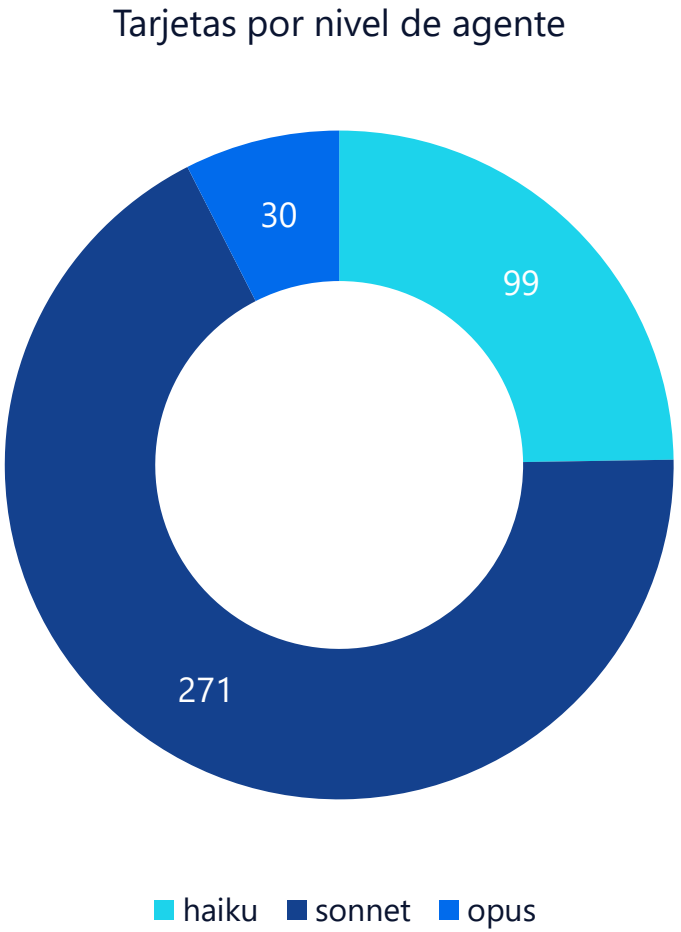
Construcción: especificación y pruebas primero

400
tarjetas de tarea

392
hechas el 2026-10-24

30
de nivel opus

- Cada comportamiento de la especificación tiene su prueba antes que el código.
- Agentes de Claude construyen cada tarjeta en su rama y su solicitud de cambios.
- Un agente verifier independiente repite las comprobaciones y publica su veredicto.
- El orquestador fusiona con la CI en verde y el veredicto PASS; la persona dueña decide la especificación y la arquitectura.



Arquitectura: Clean Architecture con puertos

Capas: las dependencias apuntan hacia dentro

presentation

servidor Fastify, interfaz web y la única raíz de composición

infrastructure

adaptadores: proveedores, guards, base de datos, archivos, language packs

application

casos de uso (scan, understand, verify, report, plan) y puertos

domain

reglas puras: afirmaciones, citas, presupuestos; no importa nada externo

dependency-cruiser comprueba la regla en cada CI: ninguna capa interior importa una exterior y no hay ciclos.

Puertos y adaptadores

Un único puerto LlmProvider; adaptadores de Ollama, OpenAI, Anthropic y uno falso para las pruebas.

Decoradores en el puerto

El guard de egreso, el de presupuesto y el esfuerzo de razonamiento envuelven cada llamada.

Raíz de composición

Solo presentation conoce las clases concretas y las inyecta en los casos de uso.

Language packs

El análisis propio de cada lenguaje (tree-sitter) queda fuera del núcleo universal.

Errores tipados

Resultados tipados y códigos RST estables; el usuario nunca ve una traza.

Prompts versionados

Prompts y formatos de tarjeta son archivos; cada ejecución registra versiones y modelos.

Stack y herramientas de calidad

Servidor

Node.js 24 LTS, TypeScript estricto, Fastify 5 y zod en las fronteras

Interfaz web

Preact y Vite; eventos del servidor (SSE) para ver los agentes en vivo

Datos

PostgreSQL 17 con SQL parametrizado y migraciones numeradas; archivos para snapshots

Modelos

SDK de OpenAI y de Anthropic, Ollama en local y un proveedor falso en las pruebas

Lectura de código

tree-sitter para C#; zip con yauzl; Excel con saxes y cfb

Despliegue

Una imagen Docker: Compose en local, Azure Container Apps, GHCR y GitHub Pages

Pruebas

Vitest para unitarias e integración; Playwright con axe para E2E y accesibilidad

Calidad y seguridad

ESLint, Prettier, dependency-cruiser, knip, CodeQL, gitleaks y OWASP ZAP

npm run verify ejecuta tipos, lint, formato, arquitectura, código muerto y pruebas con cobertura; la CI lo repite en Windows y Linux.

¿Qué lenguajes de programación entiende Rosetta?

Análisis profundo

language packs: símbolos, puntos de entrada y líneas exactas

C# y ASP.NET WebForms

.cs con tree-sitter; páginas .aspx, .ascx y .master

VB.NET

.vb, y los bloques VB dentro de páginas WebForms

SQL

.sql: tablas, vistas y procedimientos almacenados

Excel

libros subidos, convertidos a texto con sus celdas

Lectura universal

sin pack: el mapa reconoce el lenguaje y los agentes leen y citan el código

Java, Kotlin, PHP, Python, Ruby, Go

y JavaScript, TypeScript, C, C++

Legado clásico

ASP clásico, VB6, JSP, COBOL, Fortran, Pascal y Delphi

Configuración y datos

XML, JSON, YAML, archivos .config y de proyecto

Unos 45 lenguajes

por extensión; añadir uno es una línea de la tabla

Medido

C# y WebForms

- eShopLegacyWebForms con el conjunto de referencia: precisión 99,1 %, recuperación 86,7 %, citas 100 %
- VB.NET y SQL: probados con aplicaciones de ejemplo y pruebas automáticas, sin conjunto de referencia todavía
- Otro lenguaje: funciona con la lectura universal; un pack nuevo añade el análisis profundo sin tocar el núcleo

Despliegue: Azure Container Apps, España Central



Una sola imagen

La misma imagen de contenedor en local (Docker Compose) y alojada en Azure.



Sin secretos en GitHub

Login por OIDC con identidad federada; secretos solo en el almacén del entorno.



Prueba de humo superada

Salud, redirección a HTTPS, inicio de sesión de la cuenta de evaluación y un Scan.



ZAP: 0 hallazgos altos

Escaneo OWASP ZAP del commit desplegado: 0 altos, 0 medios, 0 bajos.

Imagen en producción

sha-6baab02

- Desplegada el 2026-10-24 desde main
- PostgreSQL 17 gestionado, copias con restauración a un punto de 7 días y bloqueo contra borrado
- /healthz responde 200 por HTTPS; HTTP redirige con 301
- Presupuesto con alertas de 25 USD al mes

Resultados: ejecución real en el entorno alojado

eShopLegacyWebForms (MIT), con la cuenta de evaluación y Claude Haiku 5.5 en reader, verifier y summariser, 2026-10-24

145

archivos y 42 áreas en el mapa del **Scan**, sin coste

53

tarjetas del **Understand** sobre 11 áreas

204 / 204

citas comprobadas en **Verify**

171

afirmaciones **Supported** por el verifier

0,114 USD

coste total de **Understand** y **Plan**

36

archivos en el paquete del **Plan**

Objetivos de la release 1 frente a lo medido

Métrica	Objetivo	Medida
Citas válidas en afirmaciones Supported	100 %	**100 % (204 de 204) en la demostración**
Coste de una ejecución completa de demostración	menos de 1 USD en la nube; 0 USD con Ollama	**0,114 USD con Claude Haiku 5.5**
Precisión de las afirmaciones Supported	al menos 90 % correctas	**99,1 % (98,6-100 %) de las que cubre el conjunto de referencia**
Recuperación de reglas conocidas	al menos 70 % encontradas	**86,7 % (85-90 %) de las reglas de referencia**

Medido el 2026-10-25 con el conjunto de referencia de eShopLegacyWebForms: 3 ejecuciones con Claude Haiku 5.5, 0,08 USD por ejecución. El 57 % de las afirmaciones Supported queda fuera del conjunto y no se puntúa.

Demostración en vivo

1. Iniciar sesión con la cuenta de evaluación
2. Usar el repositorio de demostración
3. **Scan**: mapa del código en segundos
4. **Understand**: elegir áreas, ver la estimación y confirmar
5. Seguir al reader y al verifier, y el coste, en vivo
6. **Plan**: preparar el brief y descargar el paquete



Rosetta

Enlaces

Aplicación alojada	https://ca-rosetta.nicewater-1ef3fc33.spaincentral.azurecontainerapps.io
Repositorio	https://github.com/blacklight0101/Rosetta
Informe de ejemplo	https://blacklight0101.github.io/Rosetta/
Vídeo	pendiente de grabar
Cuenta de evaluación	se entrega en el formulario, no en las diapositivas